TECHNICAL REPORT

Network Traffic and Protocol Analysis

OSI Model Analysis Using Wireshark

Network Protocol Analysis and OSI Model Study

Prepared by	Huzaifa Saeed
Category	Networking and Cybersecurity Fundamentals
Operating system	Windows 11
Analysis tool	Wireshark 4.6.9 with Npcap 1.88
Capture interface	Wi-Fi
Capture file	OSI-Wireshark-Capture.pcapng (262 packets, 0 dropped)
Document date	October 2026

Contents

1. Executive Summary . . . 3
Key results . . . 3
2. Objectives . . . 3
3. Environment and Tools . . . 4
4. Network Traffic Capture . . . 4
5. Understanding the OSI Model . . . 5
5.1 Layer 7 – Application . . . 5
5.2 Layer 6 – Presentation . . . 5
5.3 Layer 5 – Session . . . 5
5.4 Layer 4 – Transport . . . 5
5.5 Layer 3 – Network . . . 6
5.6 Layer 2 – Data Link . . . 6
5.7 Layer 1 – Physical . . . 6
6. Source and Destination Analysis . . . 7
7. DNS Query Analysis . . . 8
7.1 Observed DNS Packet . . . 8
7.2 Interpretation . . . 9
7.3 Display Filters . . . 9
8. HTTP Request Analysis . . . 9
8.1 Observed HTTP Packet . . . 9
8.2 Understanding the HEAD Method . . . 10
8.3 Context and Interpretation . . . 10
8.4 Cleartext HTTP . . . 10
8.5 Display Filters . . . 11
9. Packet Filtering and Analysis Techniques . . . 11
10. Observations and Findings . . . 11
11. Challenges and Practical Learning . . . 12
12. Cybersecurity Relevance . . . 12
13. Conclusion . . . 13
14. Evidence and Supporting Files . . . 13

1. Executive Summary

This project examines the Open Systems Interconnection (OSI) model through practical network traffic analysis. Rather than studying the layers only in theory, I captured live traffic on a Windows 11 system with Wireshark and inspected individual packets to identify protocols, addressing information, transport mechanisms, and application-layer data.

The analysis centred on two packets taken from a 262-packet capture: a DNS query for `example.com` (packet 42), sent over IPv6 and UDP, and an HTTP request (packet 575), sent over IPv4 and TCP to a Microsoft content-delivery host. The wider capture also showed TLS 1.2 application data, TCP connection handling, and multicast DNS traffic.

Working at the packet level gave me a practical view of how Ethernet, IP, TCP or UDP, and application protocols fit together, and introduced the display filters and dissection techniques used in network troubleshooting and security investigations.

Key results

Item	Result
Capture size	262 packets captured, 0 dropped
DNS query (packet 42)	AAAA query for example.com over UDP/IPv6, port 51208 to 53; response in packet 44
HTTP request (packet 575)	HTTP/1.1 HEAD request over TCP/IPv4, port 17065 to 80, 192.168.1.23 to 151.101.142.172
Protocols observed	Ethernet II, IPv4, IPv6, TCP, UDP, DNS, mDNS, HTTP, TLS 1.2
Security takeaway	Cleartext HTTP exposes request paths and headers; packet findings need context before they are judged

2. Objectives

- Understand the seven layers of the OSI model.
- Capture live network traffic using Wireshark.
- Identify network protocols and their corresponding layers.
- Examine source and destination IP addresses and MAC addresses.
- Understand the role of TCP and UDP ports in network communication.
- Analyse a DNS query and its corresponding response reference.
- Inspect an HTTP request and its headers.
- Apply Wireshark display filters to isolate relevant packets.
- Document packet-level observations for future networking and cybersecurity investigations.

3. Environment and Tools

Component	Details
Operating system	Windows 11
Network analysis tool	Wireshark 4.6.9
Packet capture driver	Npcap 1.88
Capture interface	Wi-Fi
Local IPv4 address	192.168.1.23 (observed in packet 575)
Local IPv6 address	fe80::411f:8ea:7a1e:3000, link-local (observed in packet 42)
Capture file	OSI-Wireshark-Capture.pcapng
Primary techniques	Packet capture, protocol dissection, display filtering
Application protocols examined	DNS and HTTP (plus TLS and mDNS observed in the packet list)
Transport protocols examined	TCP and UDP
Network protocols examined	IPv4 and IPv6

The addresses above reflect the capture environment at the time of the exercise.

4. Network Traffic Capture

Wireshark was opened on the Windows system and the active Wi-Fi interface was selected. During the capture, the system generated traffic associated with DNS resolution, HTTP, TLS-protected connections, and local-network service discovery. The packets were examined through the packet list pane and the packet details pane, by expanding protocol sections and reading fields such as addresses, ports, protocol names, DNS query data, and HTTP headers.

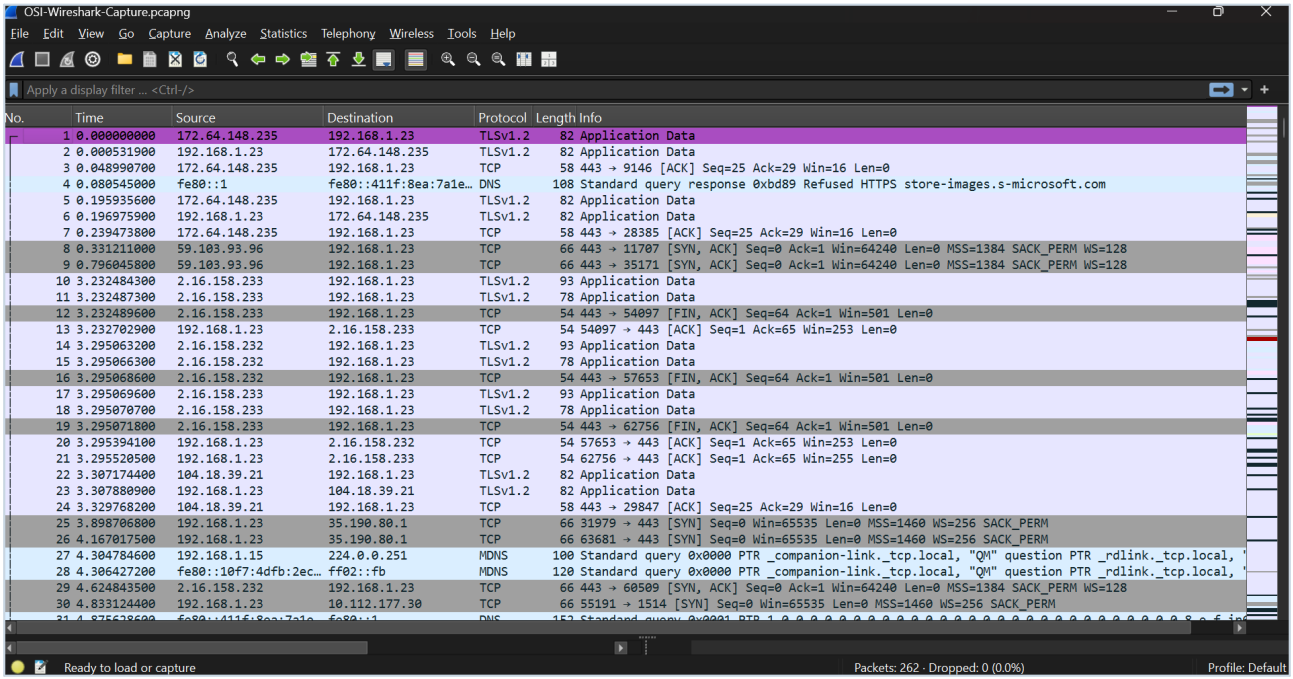


Figure 1: Captured network traffic displayed in Wireshark (262 packets, 0 dropped).

The packet list in Figure 1 shows several protocol families side by side:

- **TLSv1.2** application data records between 192.168.1.23 and remote hosts such as 172.64.148.235 and 2.16.158.233.
- **TCP** connection handling, including SYN, ACK and FIN, ACK segments (for example packets 8, 9, 12 and 25).
- **DNS**, including a response from fe80::1 with a status of Refused for an HTTPS-type query about store-images.s-microsoft.com (packet 4).
- **MDNS** queries sent to the multicast addresses 224.0.0.251 and ff02::fb (packets 27 and 28).

5. Understanding the OSI Model

The OSI model is a conceptual framework that divides network communication into seven layers, from physical transmission up to application-level services. The table below summarises what was observed at each layer; the sections that follow give the detail.

Layer	Evidence in the capture
7 – Application	DNS query (packet 42), HTTP HEAD request (packet 575), mDNS queries
6 – Presentation	TLSv1.2 records in the packet list (encryption and data protection); no separate OSI-style header
5 – Session	Connection set-up and teardown visible through TCP flags; HTTP Keep-Alive connection header; no distinct session protocol
4 – Transport	UDP 51208 to 53 (packet 42); TCP 17065 to 80 (packet 575)
3 – Network	IPv6 fe80::411f:8ea:7a1e:3000 to fe80::1 (packet 42); IPv4 192.168.1.23 to 151.101.142.172 (packet 575)
2 – Data Link	Ethernet II, source Intel_95:7e:64, destination zte_2a:dc:b8
1 – Physical	Wi-Fi radio transmission; not exposed as a header in the capture

5.1 Layer 7 – Application

The Application layer provides network services to applications. Two application protocols were examined in detail. DNS was examined through a query for `example.com`, and HTTP through a request to a Microsoft delivery endpoint. Both show how applications use the network to resolve names and request resources.

5.2 Layer 6 – Presentation

The Presentation layer concerns data representation, encoding, and encryption. In practical TCP/IP networking these functions are usually handled by applications, libraries, and protocols such as TLS rather than by a separate packet header. The packet list contains many TLSv1.2 application data records, which are commonly associated with presentation-layer functions because they protect the data carried above TCP. TLS does not map cleanly onto a single OSI layer, so this association is a conceptual one.

5.3 Layer 5 – Session

The Session layer describes establishing, managing, and ending communication sessions. In the capture, session-like behaviour appears through TCP connection set-up and teardown (SYN, SYN-ACK, FIN-ACK) and through the HTTP `Connection: Keep-Alive` header in packet 575. No distinct Session-layer protocol was identified.

5.4 Layer 4 – Transport

The Transport layer supports communication between applications. Two transport protocols were observed:

- **UDP** carried the DNS query in packet 42, from source port 51208 to destination port 53. UDP is connectionless and suits short request-response exchanges.
- **TCP** carried the HTTP request in packet 575, from source port 17065 to destination port 80, with sequence number 762 and acknowledgement number 362262. TCP provides connection-oriented, reliable delivery.

5.5 Layer 3 – Network

The Network layer provides logical addressing and forwarding between networks. Both IP versions were observed. Packet 42 used IPv6 with link-local addresses (`fe80::411f:8ea:7a1e:3000` to `fe80::1`), a next-header value of UDP (17), and a hop limit of 64. Packet 575 used IPv4, from `192.168.1.23` to `151.101.142.172`.

5.6 Layer 2 – Data Link

The Data Link layer handles delivery of frames over the local link. Both selected packets contain an Ethernet II header with the same source and destination: source `Intel_95:7e:64` (4c:1d:96:95:7e:64) and destination `zte_2a:dc:b8` (e8:43:68:2a:dc:b8). The names are Wireshark's vendor-prefix resolution of the MAC addresses. The Ethernet-style header appears even though the capture was taken on Wi-Fi because the capture driver presents frames to Wireshark in that format.

5.7 Layer 1 – Physical

The Physical layer concerns the transmission of bits through a medium. The capture used the Wi-Fi interface, which ultimately transmits data as radio signals. A standard capture on a Windows network interface does not expose radio-level details, so no physical-layer header appears in the packets.

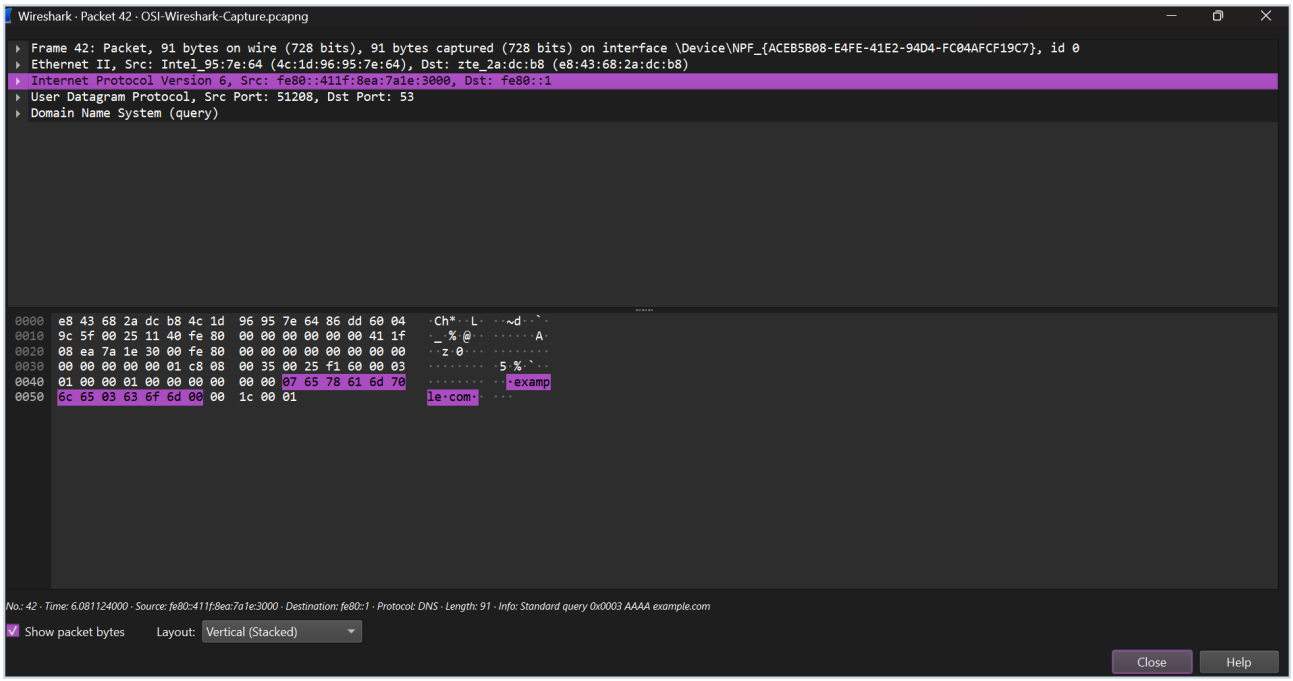


Figure 2: Packet 42 dissected into Ethernet II, IPv6, UDP, and DNS, with the "example.com" bytes highlighted in the hex view.

Figure 2 shows the layered structure directly. From the outer frame inward, packet 42 contains Ethernet II (Layer 2), Internet Protocol Version 6 (Layer 3), User Datagram Protocol (Layer 4), and a DNS query (Layer 7). The highlighted hex bytes `07 65 78 61 6d 70 6c 65 03 63 6f 6d 00` encode the name in DNS label format: a length byte, then the characters of "example", a length byte, then "com", and a zero byte to end the name.

6. Source and Destination Analysis

Source and destination fields identify the endpoints in a communication. Packet 42 (DNS over IPv6) and packet 575 (HTTP over IPv4) are compared below.

Field	Packet 42 (DNS)	Packet 575 (HTTP)
Frame length	91 bytes	409 bytes
Source MAC	Intel_95:7e:64 (4c:1d:96:95:7e:64)	Intel_95:7e:64 (4c:1d:96:95:7e:64)
Destination MAC	zte_2a:dc:b8 (e8:43:68:2a:dc:b8)	zte_2a:dc:b8 (e8:43:68:2a:dc:b8)
IP version	IPv6	IPv4
Source IP	fe80::411f:8ea:7a1e:3000	192.168.1.23
Destination IP	fe80::1	151.101.142.172
Transport	UDP	TCP
Source port	51208	17065
Destination port	53 (DNS)	80 (HTTP)
Application protocol	DNS (query)	HTTP/1.1 (HEAD request)

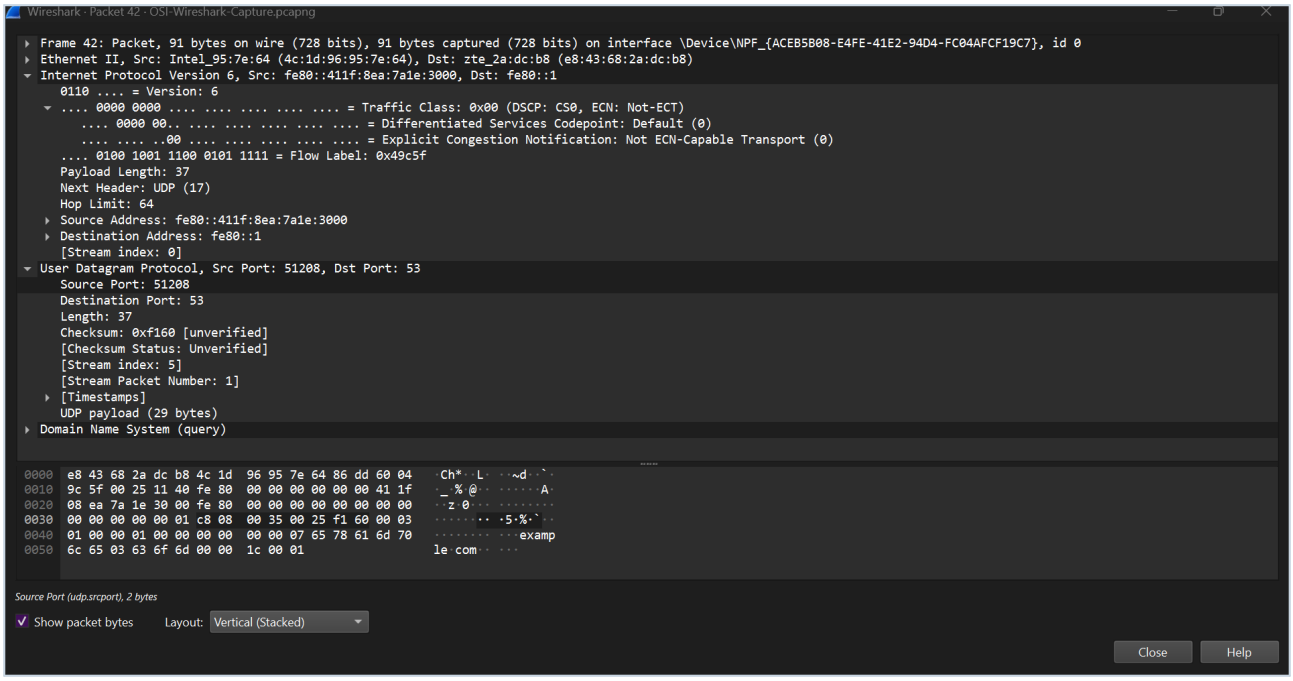


Figure 3: Packet 42 with the IPv6 and UDP sections expanded, showing addresses, ports, and header fields.

Several observations follow from these packets:

- The source IP addresses identify the local Windows system on the Wi-Fi network. The source port in each case is a high-numbered port chosen by the operating system, while the destination ports (53 and 80) are well-known service ports.
- The destination of the DNS query, `fe80::1`, is a link-local IPv6 address on the same network segment. Together with the destination MAC address belonging to a ZTE-labelled device, this is consistent with the query being sent to a local router acting as DNS resolver, although the capture alone does not confirm the device's role.

- The MAC addresses are identical in both packets because both frames travelled from the same laptop to the same next-hop device on the local link. The HTTP packet's remote IP address differs from its destination MAC address, which illustrates that IP addresses identify the end-to-end destination while MAC addresses only identify the next hop.
- In packet 42, the IPv6 header shows a payload length of 37 bytes, which equals the UDP length field. The UDP payload is 29 bytes, and the checksum is shown as unverified, which is common when checksum offloading is used on the capturing host.

7. DNS Query Analysis

DNS, the Domain Name System, translates human-readable domain names into network addresses. A DNS query for `example.com` was examined in packet 42.

7.1 Observed DNS Packet

Field	Observed value
Packet number	42 (relative time 6.081124 s)
Query name	example.com
Query type	AAAA (IPv6 address record)
Transaction ID	0x0003
Flags	0x0100, standard query
Question / answer / authority / additional counts	1 / 0 / 0 / 0
Transport protocol	UDP, source port 51208, destination port 53
Source / destination IPv6	fe80::411f:8ea:7a1e:3000 / fe80::1
Associated response	Frame 44

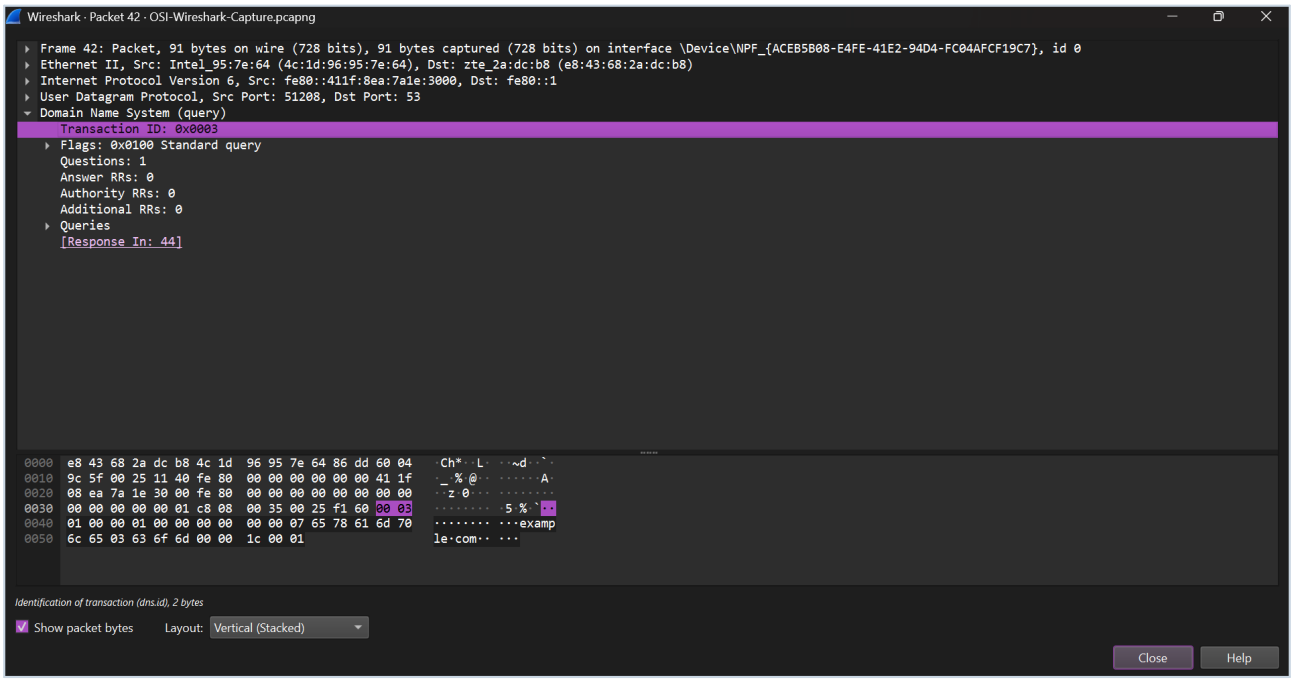


Figure 4: DNS section of packet 42 expanded, showing the transaction ID, flags, record counts, and the link to the response in frame 44.

7.2 Interpretation

The query type is AAAA, which requests the IPv6 address record for the domain. An A record would request the IPv4 address instead. The type can also be read directly from the hex bytes at the end of the packet: `00 1c` (decimal 28, the AAAA type) followed by `00 01` (class IN, the Internet class).

The flags value `0x0100` marks a standard query with the recursion-desired bit set, meaning the client asks the resolver to carry out the full lookup on its behalf. All three answer-related counters are zero because this is a query, not a response. Wireshark links the query to its response through the field `[Response In: 44]`, and the two are matched by the shared transaction ID.

DNS resolution lets applications use readable domain names instead of requiring users to enter IP addresses. The query and its response also illustrate the request-and-response pattern common to many application protocols.

7.3 Display Filters

To isolate DNS traffic: `dns`. To find queries for this domain: `dns.qry.name == "example.com"`. To show only AAAA queries: `dns.qry.type == 28`.

8. HTTP Request Analysis

HTTP is an Application-layer protocol used for communication between clients and servers. Packet 575 contains an HTTP/1.1 request sent from the local system to a Microsoft content-delivery host.

8.1 Observed HTTP Packet

Field	Observed value
Packet number	575 (409 bytes on the wire)
Source / destination IPv4	192.168.1.23 / 151.101.142.172
Transport protocol	TCP, source port 17065, destination port 80
TCP sequence / acknowledgement / length	762 / 362262 / 355 bytes
HTTP method	HEAD
HTTP version	HTTP/1.1
Host	msedge.b.tlu.dl.delivery.mp.microsoft.com
Request path	/filestreamingservice/files/... (query-string parameters redacted)
User-Agent	Microsoft BITS/7.8
Accept / Accept-Encoding	*/* / identity
Connection	Keep-Alive
Associated response	Frame 587

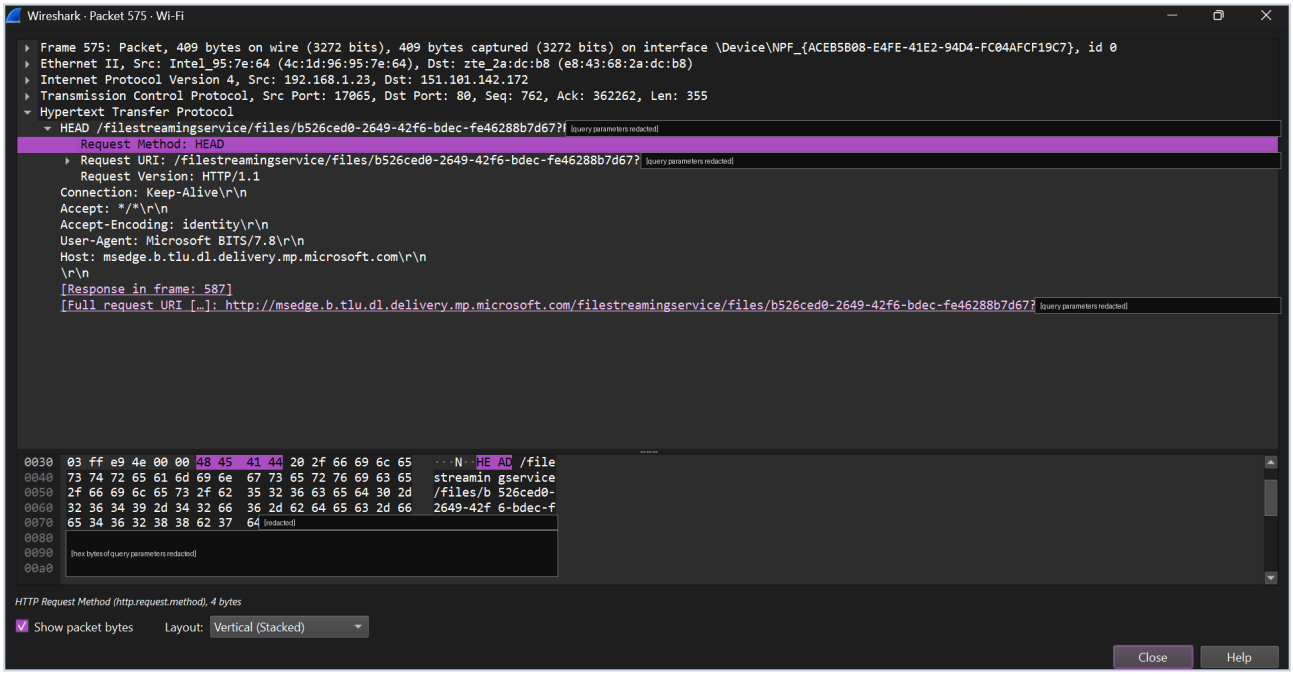


Figure 5: HTTP section of packet 575. Query-string parameters in the request URI were redacted before publication.

8.2 Understanding the HEAD Method

The request uses the HEAD method rather than GET. HEAD asks the server for the same response headers that a GET would return, but without the response body. Clients typically use it to check a resource's size, type, modification time, or availability before deciding whether to download it. A normal GET request, by contrast, asks for the resource itself.

An HTTP request is made up of several parts, all visible in Figure 5:

- **Method:** the requested operation (HEAD).
- **Request URI:** the resource being requested, here a path under /filestreamingservice/files/.
- **Host header:** the hostname the request is addressed to.
- **User-Agent header:** identifies the client software, here Microsoft BITS/7.8.
- **Accept-Encoding and Connection headers:** describe acceptable content encoding and connection handling.

8.3 Context and Interpretation

The hostname, the file-streaming path, and the BITS user-agent string are consistent with a background file-delivery request from Windows, in this case associated with a Microsoft Edge delivery host. BITS is the Background Intelligent Transfer Service, which Windows and some applications use to transfer files in the background. The request was therefore most likely generated automatically by software on the machine rather than by direct user action. The capture alone does not establish the purpose of the request or the file involved.

8.4 Cleartext HTTP

The request was decoded as HTTP and used TCP destination port 80. Traditional HTTP does not encrypt data in transit, so anyone able to capture the traffic can read the request path and headers, as demonstrated by Figure 5. HTTPS uses TLS to encrypt the HTTP exchange; the TLSv1.2 records visible in Figure 1 are an example, where the packet list shows only encrypted application data. In this capture, the request URI contained parameters that look like signed or time-limited download values.

8.5 Display Filters

To locate HTTP requests: `http.request`. To focus on this method: `http.request.method == "HEAD"`. To show traffic for this host: `ip.addr == 151.101.142.172`.

9. Packet Filtering and Analysis Techniques

Wireshark display filters help isolate relevant packets from a larger capture. The following filters are useful for repeating and extending this analysis.

Purpose	Display filter
Display DNS traffic	`dns`
Find queries for example.com	`dns.qry.name == "example.com"`
Display AAAA queries	`dns.qry.type == 28`
Display DNS responses	`dns.flags.response == 1`
Display multicast DNS	`mdns`
Display HTTP requests	`http.request`
Display HEAD requests	`http.request.method == "HEAD"`
Display TLS traffic	`tls`
Display TCP traffic	`tcp`
Display UDP traffic	`udp`
Display IPv4 traffic	`ip`
Display IPv6 traffic	`ipv6`
Display TCP SYN packets	`tcp.flags.syn == 1`
Display TCP retransmissions	`tcp.analysis.retransmission`

A display filter changes which packets are shown; it does not delete the underlying packets from the capture. This differs from a capture filter, which limits what is recorded in the first place.

10. Observations and Findings

1. Multiple protocol layers were visible in individual packets.

Packet 575 contained Ethernet II, IPv4, TCP, and HTTP sections, and packet 42 contained Ethernet II, IPv6, UDP, and DNS sections. Each packet showed how several protocols work together to carry application data.

2. Both IPv4 and IPv6 were in use.

The HTTP request used IPv4 while the DNS query used IPv6, showing that both Internet Protocol versions coexisted on the same network.

3. DNS queries carry a name, a type, and a transaction ID.

The query for example.com asked for an AAAA record, and Wireshark linked it to the response in frame 44. The packet list also showed a DNS response with a Refused status, a reminder that responses can fail and should be read, not assumed.

4. Transport ports identify communication endpoints.

The DNS query used UDP destination port 53, while the HTTP request used TCP destination port 80. Source ports were temporary high-numbered ports.

5. HTTP headers reveal application-level detail in cleartext.

The HEAD request exposed the host, resource path, and client software. This shows how Wireshark can reveal information from unencrypted application traffic.

6. Background processes generate traffic.

The HTTP request was consistent with automatic file-delivery activity from software on the machine, not a user-initiated action. Captures routinely include such traffic.

7. Packet analysis requires context.

An IP address, port, domain, or protocol on its own does not prove that traffic is malicious. Findings should be read alongside surrounding packets and system or network logs.

11. Challenges and Practical Learning

At first, the packet list and packet details panes caused some confusion. Further inspection clarified that the packet list holds the individual captured packets, while the packet details pane shows the protocol fields of the selected packet. Display filters were then used to isolate DNS and HTTP traffic.

A second lesson came from reading the packets closely: the method, query type, and header values have to be taken from the packet itself, not assumed. The HTTP request turned out to be a HEAD request, and the DNS query an AAAA query, which changed how each packet should be described. The exercise reinforced the difference between capture filters and display filters, the importance of understanding packet structure, and the need to examine multiple layers before drawing conclusions.

12. Cybersecurity Relevance

Wireshark is an important tool for network troubleshooting and security investigations. The skills developed in this exercise support activities such as:

- Investigating suspicious or unexpected DNS queries and reading query types and response status.
- Identifying unexpected network connections and their source and destination endpoints.
- Examining cleartext HTTP communication, including hosts, paths, and user-agent strings.
- Recognising automated or background traffic, such as BITS-style downloads, and separating it from suspicious activity.
- Troubleshooting TCP connectivity through connection set-up flags and retransmissions.
- Supporting network traffic analysis in a Security Operations Center (SOC).

These activities provide a foundation for more advanced work involving suspicious traffic, protocol anomalies, network scanning, and incident investigation.

13. Conclusion

This project provided hands-on experience using Wireshark to inspect real network traffic and understand the OSI model. The analysis covered DNS resolution, an HTTP request, source and destination addressing, TCP and UDP transport, and Ethernet, IPv4, and IPv6 packet information. By examining real packets rather than relying only on theory, I developed a clearer understanding of how network protocols work together during communication. The exercise also lays a foundation for further study of network troubleshooting, protocol analysis, and cybersecurity monitoring.

14. Evidence and Supporting Files

The supporting evidence consists of the saved packet capture and the five screenshots used in this report.

Figure	File	Packet	Content
1	01-network-capture.png	List view	Main Wireshark window with 262 captured packets
2	02-osi-layer-analysis.png	42	Layered dissection with "example.com" bytes highlighted
3	03-source-destination.png	42	Expanded IPv6 and UDP headers
4	04-dns-query.png	42	Expanded DNS query fields and response link
5	05-http-request.png	575	HTTP HEAD request headers (query parameters redacted)